Logo PŚ

APLIKACJE WEBOWE I SIECIOWE

Formatka dokumentacji zadania

Dokumentacja wykonania zadania

Szablon do zadań punktowanych realizowanych podczas laboratoriów

Przedmiot	Aplikacje webowe i sieciowe
Kierunek studiów	Informatyka
Forma i poziom studiów	studia stacjonarne I stopnia (inżynierskie), semestr 5
Prowadzący	dr inż. Rafał Brociek
Skład sekcji / Imię i nazwisko	Piotr Lasoń (grupa 3)
Zadanie	Laboratorium 1 – Szkielet ASP.NET Core Web API w architekturze warstwowej (CarsWebApplication), rozszerzony o pełny zestaw operacji CRUD
Liczba punktów	.....

1. Zakres wykonanego zadania

Krótko opisz, co zostało wykonane w ramach zadania. Wskaż najważniejsze elementy rozwiązania i uzyskany rezultat.

Zrealizowano szkielet backendu `CarsWebApplication` w architekturze warstwowej, zgodnie z instrukcją do Laboratorium 1. Solucja składa się z czterech projektów: `Cars.Domain` (model `Car` oraz wyliczenia `FuelType`, `BodyType`, bez zależności od innych warstw), `Cars.Application` (projekt-placeholder pod przyszłą logikę biznesową, zależny od `Cars.Domain`), `Cars.Infrastructure` (kontekst bazy danych `DataContext` oparty na EF Core + SQLite, migracja `InitialCreate` oraz dane startowe `Seed`) i `Cars.API` (projekt startowy, kontrolery REST, konfiguracja CORS i rejestracja usług w `Program.cs`).
Baza danych SQLite (`cars.db`) jest tworzona i migrowana automatycznie przy starcie aplikacji, a następnie wypełniana trzema przykładowymi samochodami, o ile tabela jest pusta.
W stosunku do zakresu podstawowego instrukcji (same odczyty), kontroler `CarsController` został rozszerzony – zgodnie z sekcją „Co dalej?" instrukcji – o pełny zestaw operacji CRUD: `GET /api/cars`, `GET /api/cars/{id}`, `POST /api/cars`, `PUT /api/cars/{id}` oraz `DELETE /api/cars/{id}`. Wszystkie endpointy zostały ręcznie przetestowane (curl oraz podgląd w przeglądarce) i działają zgodnie z oczekiwaniami, zwracając poprawne kody odpowiedzi

(200, 201, 204, 404).
Dodatkowo, poza samym dokumentem OpenAPI (`/openapi/v1.json`), w projekcie `Cars.API` skonfigurowano pakiet `Swashbuckle.AspNetCore`, dzięki czemu pod adresem `/swagger` dostępny jest interaktywny Swagger UI pozwalający przeglądać i testować wszystkie endpointy bezpośrednio z przeglądarki (zrzuty ekranu poniżej).

2. Dokumentacja rozwiązania

W razie potrzeby zamieść kluczowe fragmenty kodu, krótkie objaśnienia, zrzuty ekranu lub inne elementy potwierdzające wykonanie zadania. Nie wklejaj całego kodu źródłowego.

Architektura i zależności projektów

- `Cars.Domain` – nie zależy od żadnego projektu,
- `Cars.Application` – zależy od `Cars.Domain`,
- `Cars.Infrastructure` – zależy od `Cars.Domain`,
- `Cars.API` – zależy od `Cars.Application` oraz `Cars.Infrastructure`.

Model danych (`Cars.Domain/Car.cs`)

```
public class Car
{
    public Guid Id { get; set; }
    public string Brand { get; set; } = string.Empty;
    public string Model { get; set; } = string.Empty;
    public int DoorsNumber { get; set; }
    public int LuggageCapacity { get; set; }
    public int EngineCapacity { get; set; }
    public FuelType FuelType { get; set; }
    public DateTime ProductionDate { get; set; }
    public double AverageFuelConsumption { get; set; }
    public BodyType BodyType { get; set; }
}
```

Kontroler `CarsController` – przykładowe endpointy

Kontroler dziedziczy po `BaseApiController` (`[Route("api/[controller]")]`) i korzysta bezpośrednio z `DataContext` (zgodnie z celowym uproszczeniem przyjętym na tym etapie zajęć). Poniżej fragment ilustrujący odczyt listy oraz dodawanie rekordu – pozostałe operacje (`GetCar`, `UpdateCar`, `DeleteCar`) mają analogiczną budowę.

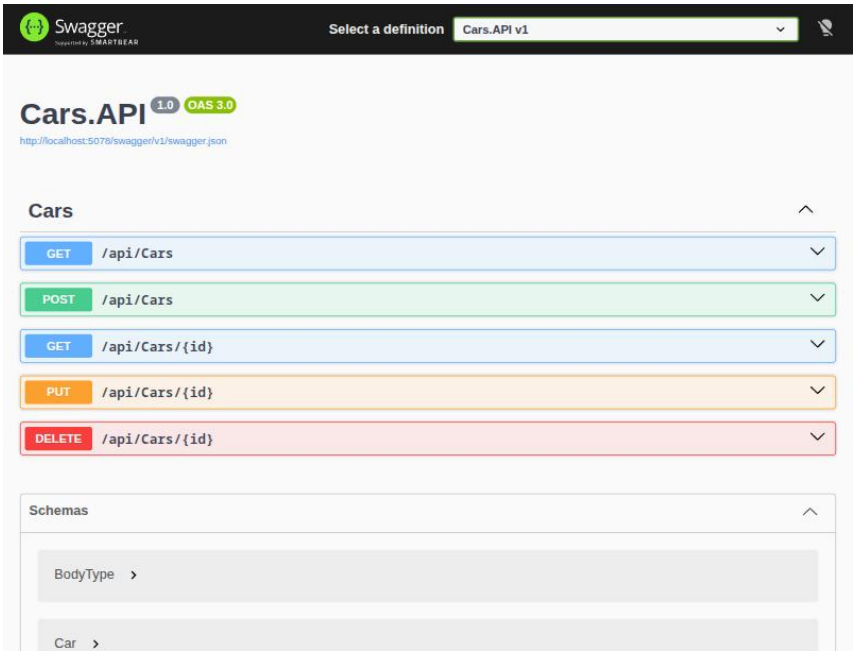
```
[HttpGet]
public async Task<ActionResult<List<Car>>> GetCars()
{
    return await _context.Cars.AsNoTracking().ToListAsync();
}

[HttpPost]
public async Task<ActionResult<Car>> CreateCar(Car car)
{
    car.Id = Guid.NewGuid();
    _context.Cars.Add(car);
    await _context.SaveChangesAsync();
    return CreatedAtAction(nameof(GetCar), new { id = car.Id }, car);
}
```

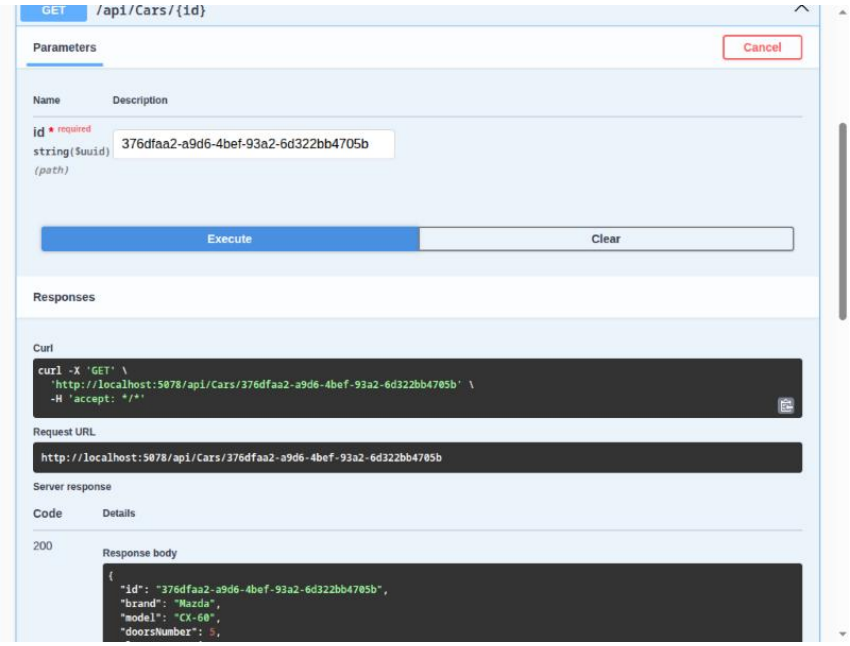
Zaimplementowane endpointy

Endpoint	Opis	Kody odpowiedzi
`GET /api/cars`	lista wszystkich samochodów	200 OK
`GET /api/cars/{id}`	pobranie jednego samochodu	200 OK / 404 Not Found
`POST /api/cars`	dodanie nowego samochodu	201 Created
`PUT /api/cars/{id}`	aktualizacja istniejącego samochodu	204 No Content / 400 / 404
`DELETE /api/cars/{id}`	usunięcie samochodu	204 No Content / 404 Not Found

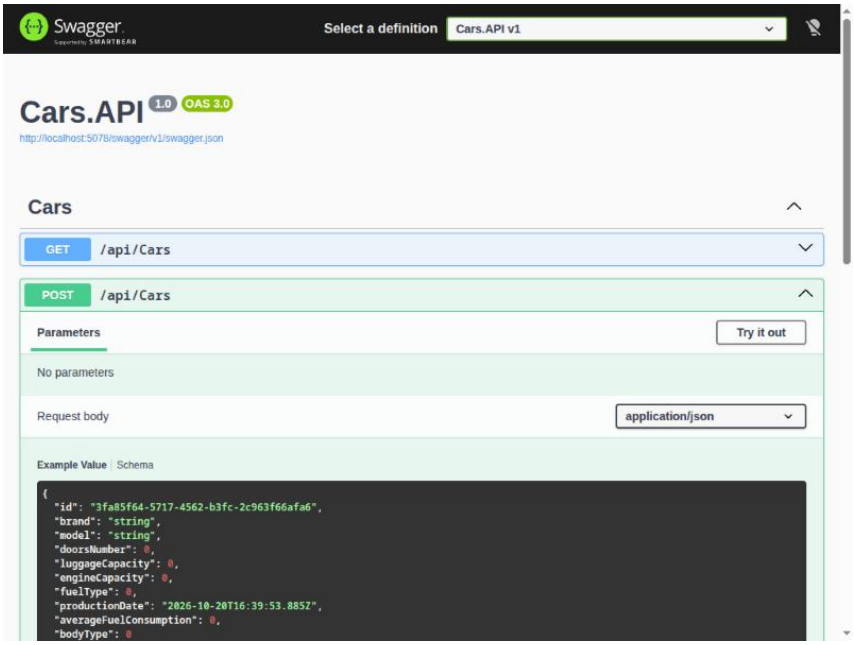
Zrzuty ekranu – Swagger UI



Rysunek 1: Swagger UI (`/swagger`) – widok wszystkich pięciu endpointów kontrolera `CarsController`: `GET`, `POST`, `GET/{id}`, `PUT/{id}`, `DELETE/{id}`.

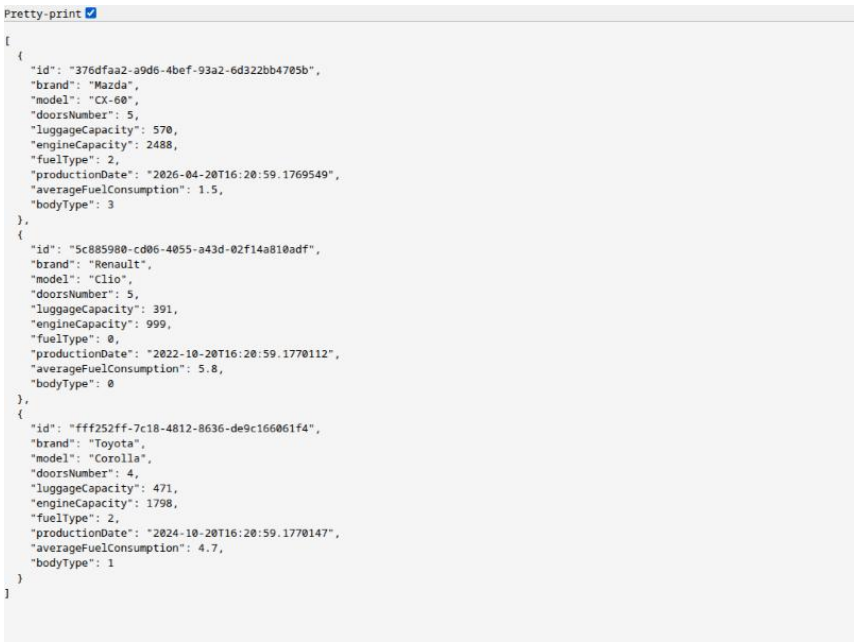


Rysunek 2: Swagger UI – wywołanie `GET /api/Cars/{id}` przez „Try it out” wraz z wygenerowanym poleceniem curl i odpowiedzią 200 z danymi samochodu.



Rysunek 3: Swagger UI – schemat treści żądania dla `POST /api/Cars` wygenerowany na podstawie modelu `Car`.

Zrzuty ekranu – surowe odpowiedzi



Rysunek 4: Odpowiedź `GET /api/cars` w przeglądarce (dane startowe wygenerowane przez `Seed.SeedData`).



Rysunek 5: Wygenerowany dokument OpenAPI (`/openapi/v1.json`) z zarejestrowanym endpointem `/api/Cars`.

3. Repozytorium GitHub

Podaj bezpośredni link do repozytorium zawierającego rozwiązanie zadania.

Link:	https://github.com/.....

Data przygotowania: 20.10.2026